

SDA

Benchmarking LLMs for Unit Test Generation from Real-World Functions

UnLeakedTestbench: il dataset che ho utilizzato io. Le funzioni di questo dataset mettono più a dura prova gli LLM testati rispetto a TestEval

Parte carina come introduzione per spiegare lo unit testing: in sostanza il lavoro umano è un bottleneck e gli LLM sono molto utili. Ma come fai a testare un LLM senza un dataset fatto bene?

Argomenta che altri benchmark non testano sullo unit level (quindi funzioni singole) ma piuttosto a livello file o classe. Aggiunge anche che spesso questi benchmark agiscono su codice non realistico che può inflazionare i risultati. Il danno più grave però è causato dai data leak a livello di addestramento. Se parti di codice sulle quali si è valutato il benchmark sono state anche utilizzate per il suo addestramento non possono essere considerate valide. Qui il mio lavoro non ha problemi proprio perché sfrutta UnLeakedTestBench per le unit da utilizzare per la generazione di test

Questi di ULT hanno candidato le funzioni da The Stack v2: hanno scelto quelle con low cyclomatic complexity (minimo 10), real-world relevance e decontamination per essere sicuro che venissero effettivamente mostrate le abilità rather than la memorization.

Al contrario, PreLeakedTestbench contiene le funzioni scartate, utile quindi per paragonare perché contiene data leak mantenendo real world relevance e cyclomatic complexity. . Our second benchmark, PLT, is a superset that contains all the functions from ULT plus all the functions that were identified as potentially contaminated. By including the decontaminated set within the leaked set, PLT represents a benchmark with mixed data quality, allowing researchers to measure the specific impact of data contamination by comparing performance against the purely decontaminated ULT.

We show that performance on ULT has a strong, positive correlation with a model's intrinsic coding ability, confirming that it measures generalization. Conversely, we demonstrate that performance on contaminated data is skewed by memorization, particularly for branch coverage.

Questo Benchmark per me è il più utile perché sto utilizzando proprio questo dataset. Potrei quindi segnalarlo come fonte 1 quando lo cito.

Tutti i paper fanno un'introduzione sullo unit testing quindi anche questo, che sottolinea come correggere un errore all'inizio è più facile e funzionale di correggerlo alla fine.

ULT aims to enhance the understanding of LLM capabilities in generating high-quality unit tests for Python code.

Spiega bene cos'è la cyclomatic complexity: fondamentale per determinare se una funzione sia utile o meno:

Given the control flow graph of a program, its cyclomatic complexity V is defined as $V = e - n + p$, where e is the number of edges, n is the number of nodes, and p is the number of connected components in the graph. A higher cyclomatic complexity generally indicates a greater number of decision points and potential execution paths within a function. We filter out functions with a cyclomatic complexity of less than 10. Sono così sicuramente non-trivial. Devo capire quale può essere il limite anche di un dataset solido come questo.

FUT: function under test deve esistere e poter essere analizzata in modo isolato: aggiunge rispetto a quello già detto prima che si elimina il rischio di allucinazione da parte dei LLM. Qui descrive come funziona il ciclo di creazione di test per la singola FUT. Sarebbe quindi utile fare un confronto con il mio processo di creazione.

Metriche:

-Test Generation Accuracy ($Pass@k$) -Code Coverage ($LCov@k$ / $BCov@k$) -Mutation Score ($Mut@k$)

Direi che quindi devo trovare altre metriche utili da aggiungere per moggarli

Hanno usato anche qui temperature=0 e max 1024 token.

Vabbe e poi nulla hanno riportato i risultati ottenuti non c'è molto altro da dire